Licenciatura en Sistemas de Información

Programación Avanzada - 2026

PROGRAMACIÓN AVANZADA · PRÁCTICA 4

Cloud Firestore: datos en tiempo real, modelo documental y reglas de seguridad

Reseñas de cursos que se actualizan en vivo en todos los navegadores, protegidas por reglas declarativas y verificadas con tests automáticos.

Dato	Detalle
Duración	60 minutos en clase + ~20 minutos de preparación previa
Modalidad	Individual o en parejas, sobre el repo de las clases anteriores
Stack	Next.js 16 (export estático) · Firebase Auth · Cloud Firestore · Emulator Suite · node:test
Costo	Cero. Firestore está incluido en el plan **Spark**: no requiere tarjeta
Requisitos	**Node.js 24 LTS** y **Java 21** o superior (el emulador de Firestore corre sobre Java)
Ejes	Modelado NoSQL · Tiempo real · Seguridad declarativa · Testing de reglas · Índices

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

1. Introducción

En la clase 2 guardamos inscripciones en **PostgreSQL** a través de SQL Connect: un modelo relacional, con esquema fijo, claves foráneas y operaciones definidas de antemano en el servidor. Hoy trabajamos con el otro gran modelo de persistencia que ofrece Firebase: **Cloud Firestore**, una base de datos **documental** pensada para que el cliente lea y escriba directamente y para **sincronizar cambios en tiempo real**.

El caso de uso es concreto: cada curso del catálogo va a tener **reseñas**. Un alumno inscripto puede dejar una puntuación de 1 a 5 y un comentario; cualquiera que esté mirando la página del curso ve aparecer la reseña nueva **sin recargar**. Es exactamente el tipo de funcionalidad donde Firestore brilla y donde una base relacional obligaría a montar WebSockets o polling.

Pero que el cliente escriba directo a la base abre la pregunta que atraviesa toda la práctica:

> Si no hay un servidor propio entre el navegador y la base de datos, ¿quién impide que alguien escriba una reseña a nombre de otro, ponga 10 estrellas o se invente la fecha? La respuesta son las Security Rules: un pequeño lenguaje declarativo que Firestore evalúa en cada lectura y escritura.

Firestore frente a SQL Connect

Aspecto	SQL Connect (clase 2)	Cloud Firestore (hoy)
Modelo	Relacional: tablas, filas, claves foráneas	Documental: colecciones de documentos JSON, subcolecciones
Esquema	Fijo, declarado en `schema.gql`	Flexible: cada documento puede tener campos distintos
Qué ejecuta el cliente	Solo operaciones con nombre definidas en el servidor	Lecturas, consultas y escrituras arbitrarias sobre rutas
Dónde vive la autorización	`@auth` en cada operación	`firestore.rules`, evaluadas en cada acceso
Relaciones	JOINs nativos	Se modelan con subcolecciones o duplicando datos (desnormalización)
Tiempo real	No nativo	`onSnapshot`: el servidor empuja los cambios
Offline	No	Caché local y compensación de latencia
Costo en Spark	Prueba de 90 días	Cuota diaria gratuita permanente

Ninguno es "mejor": son herramientas para problemas distintos. Inscripciones con cupo e integridad referencial encajan mejor en SQL; reseñas en vivo, chats, tableros colaborativos o presencia encajan mejor en Firestore. Parte del trabajo de hoy es **notar esas diferencias en la práctica**.

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

2. Objetivos de aprendizaje

1. Diseñar un **modelo documental** con subcolecciones y elegir identificadores que expresen reglas de negocio (una reseña por persona).
2. Escribir y leer datos con el SDK web, incluyendo **suscripciones en tiempo real** y su ciclo de vida en React.
3. Calcular resúmenes con **consultas de agregación** (`count`, `average`) y conocer sus límites.
4. Escribir **Security Rules** que validen identidad, forma de los datos e inmutabilidad, aplicando lista blanca.
5. Verificar las reglas con **tests automáticos** contra el emulador antes de desplegarlas.
6. Entender cuándo Firestore exige un **índice compuesto** y cómo declararlo como código.
7. Desplegar reglas e índices a producción y ajustar la **CSP**.

3. Alcance

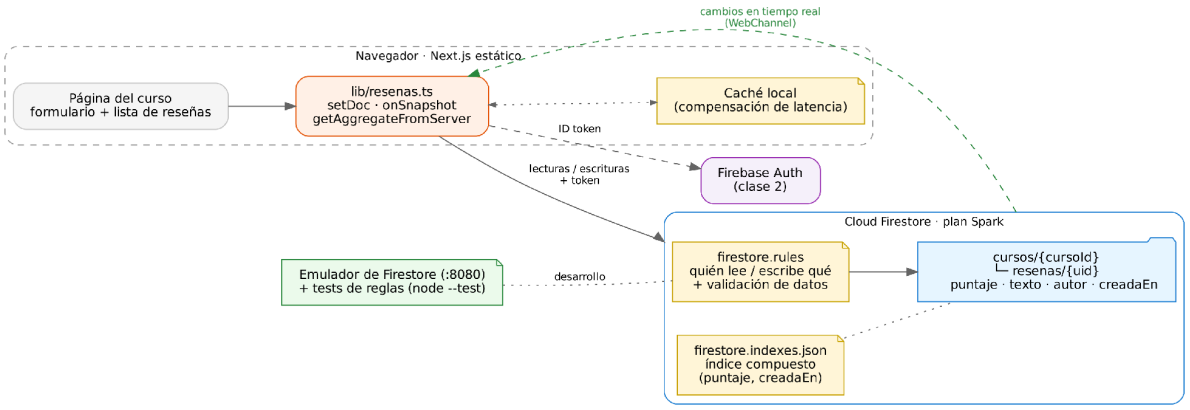
Incluye	No incluye (queda para la Práctica 5)
Alta, edición y borrado de la propia reseña	Moderación automática de contenido
Lista de reseñas en tiempo real (últimas 20)	Promedio en tiempo real guardado en el curso
Promedio y cantidad con consulta de agregación	Lógica de servidor (Cloud Functions)
Reglas de seguridad + tests de reglas	Reportes de reseñas inapropiadas
Índice compuesto para el filtro "solo 5 estrellas"	Paginación más allá de las primeras 20
Deploy de reglas e índices a producción	App Check

> **Supuestos:** la app ya tiene **Firebase Auth** con email y contraseña funcionando (clase 2) y una página por curso donde montar el componente de reseñas.
>
> `cursoId` es el identificador del curso en el catálogo (el UUID de SQL Connect o el `id` de `lib/cursos.ts`). Firestore **no necesita** que exista un documento `cursos/{cursoId}` para guardar su subcolección de reseñas.

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

4. Arquitectura



Componente	Responsabilidad
`lib/resenas.ts`	Toda la interacción con Firestore: escribir, escuchar y agregar. Los componentes no importan `firebase/firestore` directamente.
`ResenasCurso.tsx`	Formulario, lista en vivo y resumen. Maneja la suscripción con `useEffect`.
Caché local del SDK	Muestra la escritura propia al instante, antes de que el servidor confirme (compensación de latencia).
`firestore.rules`	Decide, para cada lectura y escritura, si se permite. Es el único control de seguridad real.
`firestore.indexes.json`	Índices compuestos declarados como código y versionados con el repo.
Emulador de Firestore	Base local para desarrollar y para correr los tests de reglas sin tocar producción ni consumir cuota.

5. Modelo de datos

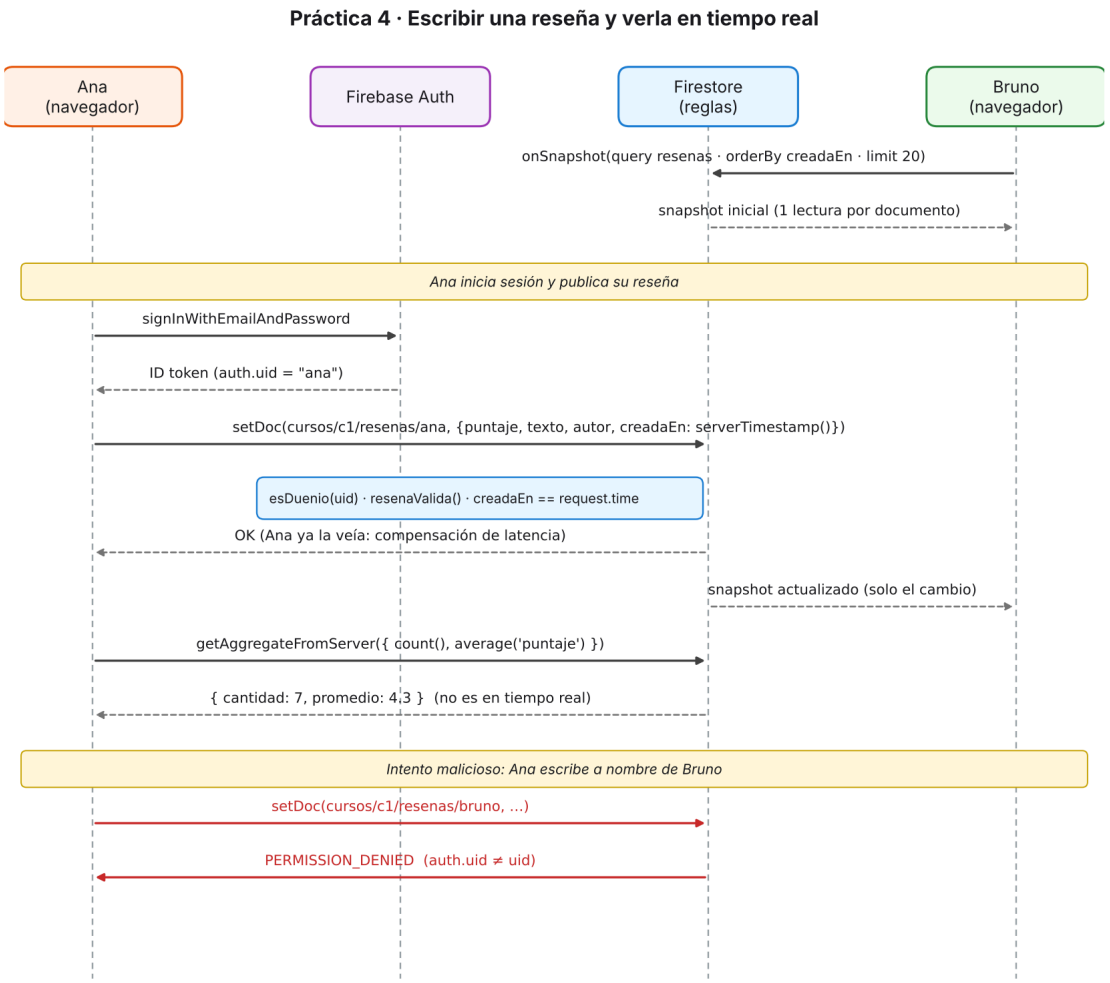
```
cursos/{cursoId}                    ← documento del curso (opcional en esta práctica)
└── resenas/{uid}                    ← subcolección; el id del documento ES el UID del autor
      puntaje   : number (int 1..5)
      texto     : string (≤ 500)
      autor     : string (≤ 80)      ← nombre para mostrar, copiado al escribir
      creadaEn  : timestamp          ← lo pone el SERVIDOR (serverTimestamp)
```

Decisión	Por qué
Subcolección dentro del curso	Las reseñas siempre se consultan por curso. La ruta ya filtra: no hace falta un campo `cursoId` ni un índice para buscarlo.

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

Decisión	Por qué
Id del documento = UID	"Una reseña por persona y curso" queda garantizado por la estructura: escribir de nuevo actualiza la misma reseña. Además, las reglas pueden comparar el id con `request.auth.uid`.
`autor` duplicado en la reseña	Desnormalización deliberada: mostrar la lista no requiere una lectura extra por reseña para buscar el nombre. El costo: si alguien cambia su nombre, sus reseñas viejas conservan el anterior.
`creadaEn` con `serverTimestamp()`	El reloj del cliente no es confiable (puede estar mal o ser manipulado). Las reglas exigen que coincida con `request.time`.

6. Flujo en tiempo real



Tres detalles del diagrama que conviene observar en clase:

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

- **Compensación de latencia:** Ana ve su reseña antes de que el servidor confirme. Si las reglas la rechazan, el SDK la retira de la lista y la promesa de `setDoc` falla.
- **El snapshot de Bruno trae solo el cambio**, pero el snapshot inicial cuesta una lectura por documento. Con `limit(20)`, cada visita a la página cuesta como máximo 20 lecturas.
- **La agregación no es en tiempo real:** `getAggregateFromServer` no admite listeners. En la Práctica 5 lo resolvemos manteniendo el resumen desde el servidor.

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

7. Preparación previa

En la consola de Firebase

1. **Firestore Database** → *Crear base de datos* → edición **Standard**.
2. Ubicación: elegí una y anotala. **No se puede cambiar después.** `southamerica-east1` (San Pablo) da menor latencia desde Argentina; `nam5` es la opción multirregión por defecto.
3. Modo: **producción** (todo cerrado). Las reglas reales las desplegamos nosotros en el Paso 8.

En tu máquina

1. Verificá `node -v` (24.x) y `java -version` (21 o superior). Sin Java, el emulador de Firestore no arranca.
2. `npx firebase init firestore`: aceptá los nombres por defecto `firestore.rules` y `firestore.indexes.json`. El comando agrega la clave `firestore` a tu `firebase.json`.
3. En `firebase.json`, dentro de `emulators`, agregá `"firestore": { "port": 8080 }` (junto a `auth` de la clase 2).
4. Instalá la librería de tests de reglas: `npm i -D @firebase/rules-unit-testing@6`.
5. En `package.json`, agregá el script `"test:reglas": "firebase emulators:exec --only firestore \"node --test tests/\""`.
6. Probá `npx firebase emulators:start --only auth,firestore`: la UI en `http://127.0.0.1:4000` tiene que mostrar la pestaña Firestore.

> **Llegás a clase con:** la base de datos creada en la consola, los emuladores de Auth y Firestore arrancando y `@firebase/rules-unit-testing` instalado.

8. Cronograma

Minuto	Pasos	Actividad
00 – 05	—	Introducción: documental vs. relacional, modelo de datos, la pregunta de seguridad.
05 – 20	1 – 4	Cliente: inicialización, escribir reseña, escuchar en tiempo real. Probar con dos navegadores.
20 – 25	5	Resumen con agregación.
25 – 45	6 – 7	Reglas de seguridad y tests contra el emulador hasta que todo pase.
45 – 52	8	Índice compuesto para el filtro de 5 estrellas.
52 – 57	9	Deploy de reglas e índices, CSP, PR.
57 – 60	—	Cierre: cuotas, qué falta para tener un promedio en tiempo real (puente a la Práctica 5).

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

9. Paso a paso

Trabajá con tres terminales: `npx firebase emulators:start --only auth,firestore`, `npm run dev` y una libre para los tests.

Paso 1 · Inicializar Firestore en el cliente

Creá `lib/resenas.ts`. Igual que con Auth y SQL Connect, en desarrollo el SDK se conecta al emulador local.

```
// lib/resenas.ts — Práctica 4 (Firestore). Capa de acceso a datos de reseñas.
import { getApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import {
  average,
  collection,
  connectFirestoreEmulator,
  count,
  deleteDoc,
  doc,
  getAggregateFromServer,
  getFirestore,
  limit,
  onSnapshot,
  orderBy,
  query,
  serverTimestamp,
  setDoc,
  where,
  type Firestore,
  type Timestamp,
  type Unsubscribe,
} from "firebase/firestore";

export interface Resena {
  uid: string; // = id del documento
  puntaje: number; // 1..5
  texto: string;
  autor: string;
  creadaEn: Timestamp | null; // null mientras el serverTimestamp está pendiente (escritura local)
}

let db: Firestore | null = null;
export function firestore(): Firestore {
  if (db) return db;
  db = getFirestore(getApp());
  if (process.env.NODE_ENV === "development") connectFirestoreEmulator(db, "127.0.0.1", 8080);
  return db;
}

const resenasDe = (cursoId: string) => collection(firestore(), "cursos", cursoId, "resenas");
```

Verificación: `npm run typecheck` sin errores.

Paso 2 · Montar el componente

Creá `app/components/ResenasCurso.tsx` y usalo en la página o tarjeta del curso, pasándole `cursoId` y el `uid` del usuario actual (de `useUsuario()`, clase 2).

```
"use client";
// app/components/ResenasCurso.tsx — Práctica 4. Formulario + lista en tiempo real + resumen.
```

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

```
import { useEffect, useState, type FormEvent } from "react";
import { escucharResenas, guardarResena, resumenDe, type Resena } from "@/lib/resenas";

export default function ResenasCurso({ cursoId, uid }: { cursoId: string; uid: string | null }) {
  const [resenas, setResenas] = useState<Resena[]>([]);
  const [resumen, setResumen] = useState<{ cantidad: number; promedio: number | null } | null>(null);
  const [soloCinco, setSoloCinco] = useState(false);
  const [error, setError] = useState<string | null>(null);

  // Suscripción en tiempo real: se cancela al desmontar o al cambiar de curso/filtro.
  useEffect(() => {
    return escucharResenas(cursoId, setResenas, (e) => setError(e.message), soloCinco);
  }, [cursoId, soloCinco]);

  useEffect(() => {
    resumenDe(cursoId).then(setResumen).catch((e: Error) => setError(e.message));
  }, [cursoId, resenas.length]); // recalcular cuando cambia la cantidad (no es en tiempo real)

  const mia = resenas.find((r) => r.uid === uid);

  async function onSubmit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    const f = new FormData(e.currentTarget);
    try {
      await guardarResena(cursoId, Number(f.get("puntaje")), String(f.get("texto")), !mia);
      setError(null);
    } catch (err) {
      setError((err as Error).message);
    }
  }

  return (
    <section>
      <h3>
        Reseñas {resumen && resumen.cantidad > 0 && `· ★ ${resumen.promedio?.toFixed(1)} (${resumen.cantidad})`}
      </h3>
      <label>
        <input type="checkbox" checked={soloCinco} onChange={(e) => setSoloCinco(e.target.checked)} /> Solo 5
estrellas
      </label>
      {uid && (
        <form onSubmit={onSubmit}>
          <select name="puntaje" defaultValue={mia?.puntaje ?? 5}>
            {[5, 4, 3, 2, 1].map((n) => (
              <option key={n} value={n}>{"★".repeat(n)}</option>
            ))}
          </select>
          <textarea name="texto" maxLength={500} defaultValue={mia?.texto ?? ""} required />
          <button type="submit">{mia ? "Actualizar mi reseña" : "Publicar"}</button>
        </form>
      )}
      {error && <p role="alert">{error}</p>}
      <ul>
        {resenas.map((r) => (
          <li key={r.uid}>
            <strong>{"★".repeat(r.puntaje)}</strong> {r.texto} — <em>{r.autor}</em>
          </li>
        ))}
      </ul>
    </section>
  );
}
```

Ciclo de vida: `onSnapshot` abre una conexión que queda viva. El `useEffect` devuelve la función de cancelación para que React la llame al desmontar o al cambiar `cursoId` o el filtro. Sin eso, cada navegación deja una suscripción huérfana que sigue consumiendo lecturas.

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

Paso 3 · Escribir la reseña

```
/** Paso 3 · Crear o editar MI reseña. El id del documento es mi UID: una reseña por persona y curso. */
export async function guardarResena(cursoId: string, puntaje: number, texto: string, esNueva: boolean):
Promise<void> {
  const usuario = getAuth().currentUser;
  if (!usuario) throw new Error("Tenés que ingresar para dejar una reseña.");
  const ref = doc(resenasDe(cursoId), usuario.uid);
  const datos = { puntaje, texto, autor: usuario.displayName ?? "Anónimo" };
  if (esNueva) await setDoc(ref, { ...datos, creadaEn: serverTimestamp() });
  else await setDoc(ref, datos, { merge: true }); // conserva creadaEn
}

export async function borrarResena(cursoId: string): Promise<void> {
  const usuario = getAuth().currentUser;
  if (!usuario) return;
  await deleteDoc(doc(resenasDe(cursoId), usuario.uid));
}
```

- `doc(resenas, usuario.uid)`: el id es el UID, así que "crear" y "editar" apuntan al mismo documento.
- Al crear se manda `creadaEn: serverTimestamp()`. Al editar se usa `{ merge: true }` sin `creadaEn`, para conservar la fecha original (las reglas del Paso 6 lo exigen).

Verificación: ingresá en la app, publicá una reseña y miralas en la UI de emuladores (`127.0.0.1:4000` → Firestore → `cursos/{id}/resenas`).

Paso 4 · Escuchar en tiempo real

```
/** Paso 4 · Últimas reseñas en tiempo real. Devuelve la función para cancelar la suscripción. */
export function escucharResenas(
  cursoId: string,
  alCambiar: (resenas: Resena[]) => void,
  alFallar: (e: Error) => void,
  soloCincoEstrellas = false,
): Unsubscribe {
  const filtros = soloCincoEstrellas ? [where("puntaje", "==", 5)] : [];
  const q = query(resenasDe(cursoId), ...filtros, orderBy("creadaEn", "desc"), limit(20));
  return onSnapshot(
    q,
    (snap) =>
      alCambiar(
        snap.docs.map((d) => {
          const data = d.data({ serverTimestamps: "estimate" });
          return { uid: d.id, puntaje: data.puntaje, texto: data.texto, autor: data.autor, creadaEn: data.creadaEn
?? null };
        }),
      ),
    alFallar,
  );
}
```

- `data({ serverTimestamps: "estimate" })`: mientras la escritura local está pendiente, `creadaEn` todavía no tiene valor del servidor; con `estimate` el SDK usa una estimación en lugar de `null`.
- El segundo callback de `onSnapshot` recibe errores: por ejemplo, un índice faltante (Paso 8) o un permiso denegado.

Verificación: abrí la página en dos navegadores (uno en modo incógnito, con otra cuenta). Publicá desde uno: la reseña aparece en el otro sin recargar.

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

Paso 5 · Resumen con consulta de agregación

```
/** Paso 5 · Promedio y cantidad calculados por el servidor (consulta de agregación). */
export async function resumenDe(cursoId: string): Promise<{ cantidad: number; promedio: number | null }> {
  const snap = await getAggregateFromServer(resenasDe(cursoId), { cantidad: count(), promedio: average("puntaje")
});
  return snap.data();
}
```

La agregación se calcula en el servidor sin descargar los documentos: se cobra **una lectura por cada lote de hasta 1.000 entradas de índice** leídas. Su límite: **no funciona con listeners en tiempo real ni offline**. Por eso el componente la vuelve a pedir cuando cambia la cantidad de reseñas.

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

Paso 6 · Reglas de seguridad

Reemplazá el contenido de `firestore.rules`:

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {

    // ── Funciones auxiliares ──────────────────────────────────────────
    function autenticado() {
      return request.auth != null;
    }

    function esDuenio(uid) {
      return autenticado() && request.auth.uid == uid;
    }

    // Lista blanca de campos, tipos y rangos. Todo lo que no está acá, se rechaza.
    function resenaValida() {
      let d = request.resource.data;
      return d.keys().hasOnly(['puntaje', 'texto', 'autor', 'creadaEn'])
        && d.keys().hasAll(['puntaje', 'texto', 'autor', 'creadaEn'])
        && d.puntaje is int && d.puntaje >= 1 && d.puntaje <= 5
        && d.texto is string && d.texto.size() <= 500
        && d.autor is string && d.autor.size() <= 80;
    }

    // ── Cursos: solo lectura desde el cliente ─────────────────────────
    match /cursos/{cursoId} {
      allow read: if true;
      allow write: if false;

      // ── Reseñas: el id del documento es el UID del autor ────────────
      match /resenas/{uid} {
        allow read: if true;

        allow create: if esDuenio(uid)
          && resenaValida()
          && request.resource.data.creadaEn == request.time;   // la fecha la pone el servidor

        allow update: if esDuenio(uid)
          && resenaValida()
          && request.resource.data.creadaEn == resource.data.creadaEn;  // inmutable

        allow delete: if esDuenio(uid);
      }
    }

    // Todo lo demás: cerrado.
    match /{documento=**} {
      allow read, write: if false;
    }
  }
}
```

Regla	Qué garantiza
`esDuenio(uid)`	Solo podés escribir el documento cuyo id es tu UID: nadie escribe a nombre de otro.

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

Regla	Qué garantiza
`keys().hasOnly([...])`	**Lista blanca**: cualquier campo no previsto (`destacada`, `verificada`…) hace fallar la escritura.
`puntaje is int && >= 1 && <= 5`	Tipo y rango. `4.5` o `"5"` se rechazan.
`creadaEn == request.time`	Al crear, la fecha tiene que ser exactamente la del servidor (`serverTimestamp()`).
`creadaEn == resource.data.creadaEn`	Al editar, la fecha no cambia. `resource` es el documento actual y `request.resource` el resultante.
`match /cursos/{cursoId}` → `write: if false`	El cliente no puede escribir el documento del curso (en la Práctica 5 lo escribe el servidor).
`match /{documento=**}` al final	Todo lo que no se habilitó explícitamente, queda cerrado.

Las reglas no son filtros: si una consulta *podría* devolver un documento que el usuario no tiene permiso de leer, Firestore rechaza la consulta entera, en lugar de devolver solo los permitidos. Lo vamos a necesitar en la Práctica 5.

Paso 7 · Tests de reglas contra el emulador

Creá `tests/reglas.test.mjs`. Cada test usa un contexto autenticado (o anónimo) y afirma si la operación **debe** funcionar o fallar.

```
// tests/reglas.test.mjs — Práctica 4. Se ejecuta contra el emulador de Firestore:
//   npx firebase emulators:exec --only firestore "node --test tests/"
import { after, before, beforeEach, test } from "node:test";
import { readFileSync } from "node:fs";
import { assertFails, assertSucceeds, initializeTestEnvironment } from "@firebase/rules-unit-testing";
import { doc, getDoc, serverTimestamp, setDoc, Timestamp, updateDoc } from "firebase/firestore";

/** @type {import('@firebase/rules-unit-testing').RulesTestEnvironment} */
let env;
const CURSO = "seguridad-web";
const resena = (extra = {}) => ({ puntaje: 5, texto: "Muy bueno", autor: "Ana", creadaEn: serverTimestamp(),
...extra });

before(async () => {
  env = await initializeTestEnvironment({
    projectId: "demo-resenas",
    firestore: { rules: readFileSync("firestore.rules", "utf8"), host: "127.0.0.1", port: 8080 },
  });
});
beforeEach(() => env.clearFirestore());
after(() => env.cleanup());

test("cualquiera puede leer reseñas, incluso sin login", async () => {
  const anon = env.unauthenticatedContext().firestore();
  await assertSucceeds(getDoc(doc(anon, "cursos", CURSO, "resenas", "ana")));
});

test("Ana puede crear SU reseña", async () => {
  const db = env.authenticatedContext("ana").firestore();
  await assertSucceeds(setDoc(doc(db, "cursos", CURSO, "resenas", "ana"), resena()));
});

test("Ana NO puede crear una reseña a nombre de Bruno", async () => {
```

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

```
  const db = env.authenticatedContext("ana").firestore();
  await assertFails(setDoc(doc(db, "cursos", CURSO, "resenas", "bruno"), resena()));
});

test("sin login no se puede escribir", async () => {
  const db = env.unauthenticatedContext().firestore();
  await assertFails(setDoc(doc(db, "cursos", CURSO, "resenas", "ana"), resena()));
});

test("puntaje fuera de rango o con decimales se rechaza", async () => {
  const db = env.authenticatedContext("ana").firestore();
  const ref = doc(db, "cursos", CURSO, "resenas", "ana");
  await assertFails(setDoc(ref, resena({ puntaje: 6 })));
  await assertFails(setDoc(ref, resena({ puntaje: 4.5 })));
});

test("no se pueden agregar campos extra (p. ej. destacada: true)", async () => {
  const db = env.authenticatedContext("ana").firestore();
  await assertFails(setDoc(doc(db, "cursos", CURSO, "resenas", "ana"), resena({ destacada: true })));
});

test("la fecha de creación la pone el servidor y no se puede cambiar", async () => {
  const db = env.authenticatedContext("ana").firestore();
  const ref = doc(db, "cursos", CURSO, "resenas", "ana");
  await assertFails(setDoc(ref, resena({ creadaEn: Timestamp.fromDate(new Date(2020, 0, 1)) })));
  await assertSucceeds(setDoc(ref, resena()));
  await assertFails(updateDoc(ref, { creadaEn: Timestamp.now() }));
  await assertSucceeds(updateDoc(ref, { puntaje: 4 }));
});

test("el cliente no puede escribir el documento del curso", async () => {
  const db = env.authenticatedContext("ana").firestore();
  await assertFails(setDoc(doc(db, "cursos", CURSO), { resumen: { cantidad: 999, promedio: 5 } }));
});
```

Ejecutalo con `npm run test:reglas`. El comando levanta un emulador de Firestore efímero, corre los tests y lo apaga.

> **Hacelo fallar a propósito:** cambiá en las reglas `puntaje <= 5` por `puntaje <= 6` y volvé a correr los tests. Si ningún test se pone rojo, tus tests no están probando lo que creés. Un test de seguridad que no puede fallar no vale nada.

Verificación: los 8 tests en verde.

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

Paso 8 · Índice compuesto

Marcá *Solo 5 estrellas* en la app. La consulta combina un filtro de igualdad (`puntaje == 5`) con un orden por **otro** campo (`creadaEn`), y eso requiere un **índice compuesto**. En producción la consulta falla con `FAILED_PRECONDITION` y un mensaje que incluye un link a la consola para crearlo.

En lugar de crearlo con clicks, declaralo como código en `firestore.indexes.json`:

```
{
  "indexes": [
    {
      "collectionGroup": "resenas",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "puntaje", "order": "ASCENDING" },
        { "fieldPath": "creadaEn", "order": "DESCENDING" }
      ]
    }
  ],
  "fieldOverrides": []
}
```

Los índices de un solo campo los crea Firestore automáticamente; los compuestos hay que declararlos. Tenerlos en el repo hace que el entorno sea reproducible y que el cambio pase por revisión.

> **Ojo con el emulador:** el emulador de Firestore **no exige índices**, así que la consulta funciona en desarrollo aunque falte el índice. Este error solo aparece en producción. Es una diferencia real entre emulador y servicio, y la razón por la que los índices tienen que estar declarados en el repo.

Paso 9 · Producción

1. Probá las reglas una última vez: `npm run test:reglas`.
2. Desplegá reglas e índices: `npx firebase deploy --only firestore:rules,firestore:indexes`. El índice tarda unos minutos en construirse (consola → Firestore → Índices).
3. **CSP:** agregá `https://firestore.googleapis.com` a `connect-src` en `firebase.json`. Sin eso, la app de producción no puede abrir la conexión en tiempo real.
4. Rama, PR, preview, merge: como siempre.

Verificación: en producción, con dos navegadores, una reseña nueva aparece en ambos. Desde la consola del navegador, intentá escribir en la reseña de otra persona: tiene que fallar con `permission-denied`.

10. Cuotas y costos en el plan Spark

Recurso	Cuota gratuita	Cómo la consume esta app
Lecturas	50.000 por día	Cada visita: hasta 20 (snapshot inicial) + 1 por agregación + 1 por cada reseña nueva mientras la página está abierta

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

Recurso	Cuota gratuita	Cómo la consume esta app
Escrituras	20.000 por día	1 por reseña creada o editada
Borrados	20.000 por día	1 por reseña borrada
Almacenamiento	1 GB	Una reseña pesa menos de 1 KB

Al superar la cuota diaria, Firestore deja de responder hasta el reinicio del día siguiente, pero nunca cobra. El `limit(20)` no es un detalle estético: sin él, una página con 5.000 reseñas costaría 5.000 lecturas por visita.

11. Preguntas de reflexión

Respondelas en `REFLEXION.md`, en una sección nueva `## Práctica 4 · Firestore`:

1. ¿Por qué el id del documento es el UID? ¿Qué tendrías que agregar (en reglas o en código) si usaras ids aleatorios con `addDoc`?
2. La reseña guarda una copia del nombre del autor. ¿Qué ganás y qué perdés con esa desnormalización? ¿Cómo resolverías el cambio de nombre?
3. ¿Qué diferencia hay entre `resource.data` y `request.resource.data`? Dá un ejemplo de regla que use ambos.
4. ¿Por qué la regla de creación exige `creadaEn == request.time` en lugar de confiar en la fecha del cliente?
5. ¿Qué limitación tiene `getAggregateFromServer` y cómo afecta la experiencia de usuario? ¿Qué alternativas hay?
6. Para inscripciones con cupo máximo, ¿usarías Firestore o SQL Connect? Justificá con lo que viste en las dos prácticas.

12. Entregables

#	Entregable	Formato
1	`lib/resenas.ts`, `ResenasCurso.tsx`, `firestore.rules`, `firestore.indexes.json` y `tests/`	URL del repo
2	Salida de `npm run test:reglas` (todos en verde) y la prueba de "hacerlo fallar"	Captura en `REFLEXION.md`
3	Video corto o dos capturas mostrando la actualización en tiempo real entre dos navegadores	Enlace o imágenes
4	PR mergeado con reglas, índice y CSP	URL
5	Respuestas a la reflexión	`REFLEXION.md`

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

13. Rúbrica de evaluación

Criterio	Pts	Insuficiente	Esperado	Destacado
Modelo y escritura	15	Ids aleatorios o fecha del cliente.	Subcolección, id = UID, `serverTimestamp`, edición conserva la fecha.	Justifica la desnormalización y propone cómo actualizar nombres.
Tiempo real	20	Sin suscripción o con suscripciones huérfanas.	`onSnapshot` con `limit`, cancelación en el `useEffect`, errores manejados.	Muestra estados de escritura pendiente (`hasPendingWrites`).
Reglas	25	Reglas abiertas o que se pueden eludir.	Identidad, lista blanca, tipos, rangos e inmutabilidad de la fecha.	Agrega límites adicionales justificados (p. ej., tamaño del nombre, borrado solo del dueño).
Tests de reglas	20	Sin tests o tests que no fallan nunca.	Los 8 casos en verde y evidencia de haberlos hecho fallar.	Tests adicionales propios (p. ej., edición de reseña ajena).
Índices y producción	10	Índice creado a mano o CSP rota.	Índice en `firestore.indexes.json`, deploy y CSP correctos.	Explica la diferencia emulador / producción con los índices.
Reflexión	10	Incompleta.	Todas respondidas con fundamentos.	Compara con SQL Connect con ejemplos concretos.

Total: 100 puntos. Se aprueba con 60, con *Reglas* y *Tests de reglas* al menos en nivel *Esperado*.

Anexo A · Problemas frecuentes

Síntoma	Causa probable
El emulador de Firestore no arranca	Falta Java 21+ (`java -version`), o el puerto 8080 está ocupado.
`permission-denied` al publicar	Las reglas lo rechazan: falta sesión, el id no es tu UID, sobra un campo o `creadaEn` no es `serverTimestamp()`.
`permission-denied` al editar	Mandaste `creadaEn` nuevo, o hiciste `setDoc` sin `merge` y sin la fecha.
La lista no se actualiza en el otro navegador	Suscripción cancelada o no creada: revisá el `useEffect` y su arreglo de dependencias.

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

Síntoma	Causa probable
The query requires an index (solo en producción)	Falta el índice compuesto o todavía se está construyendo.
Producción: *Refused to connect*	Falta `https://firestore.googleapis.com` en `connect-src`.
`rules-unit-testing` pide otra versión de Node	La v6 requiere Node 24.12+. Con Node 22, usá la v5 en un `package.json` aparte (requiere `firebase@12`).

Anexo B · Referencias

- Cloud Firestore — Modelo de datos: firebase.google.com/docs/firestore/data-model
- Escuchar cambios en tiempo real: firebase.google.com/docs/firestore/query-data/listen
- Consultas de agregación: firebase.google.com/docs/firestore/query-data/aggregation-queries
- Security Rules — Condiciones y validación de datos: firebase.google.com/docs/firestore/security/rules-conditions
- Probar reglas con el emulador: firebase.google.com/docs/rules/unit-tests
- Índices: firebase.google.com/docs/firestore/query-data/indexing
- Precios y cuotas: firebase.google.com/docs/firestore/pricing